

search documentation

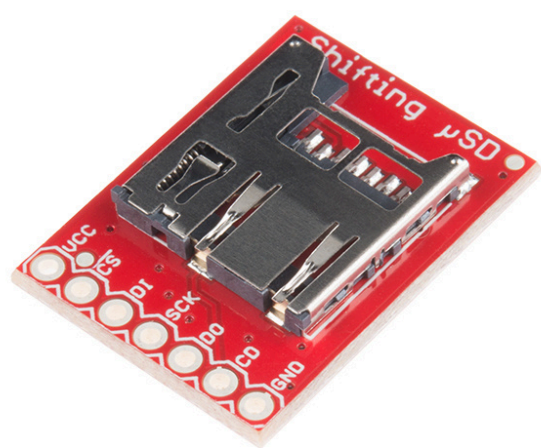


MicroSD Breakout With Level Shifter Hookup Guide

CONTRIBUTORS:  .BRENT.

Introduction

The MicroSD Breakout with Built-In Level Shifter makes it easy to add mass storage to your project while working with a 5V system.



SparkFun Level Shifting microSD Breakout
DEV-13743
\$6.50

Required Materials

To follow along with this hookup guide, you will need the following:

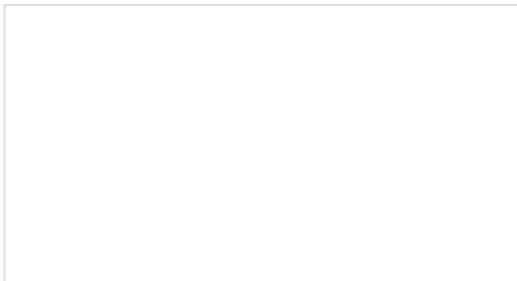
Shifting μSD Hookup Guide	
	SparkFun Level Shifting microSD Breakout DEV-13743 The SparkFun Level Shifting microSD Breakout is quite similar to the SparkFun microSD Transflash Breakout, but with t...
	USB Micro-B Cable - 6 Foot CAB-10215 USB 2.0 type A to micro USB 5-pin. This is a new, smaller connector for USB devices. Micro USB connectors are about...
microSD USB Reader	

	COM-13004 This is an awesome little microSD USB reader. Just slide your microSD card into the inside of the USB connector, then s...
	microSD Card with Adapter - 32GB (Class 10)
	COM-14832 SparkFun RedBoard Qwiic DEV-15123
	Straight Header - Female (PTH, 0.1in., 40-Pin) PRT-00115 Single row of 40-holes, female header. Can be cut to size with a pair of wire-cutters. Standard .1" spacing. We use them...
	Jumper Wires Premium 6" M/M Pack of 10 PRT-08431 This is a SparkFun exclusive! These are 155mm long jumpers with male connectors on both ends. Use these to jumper f...

Suggested Reading

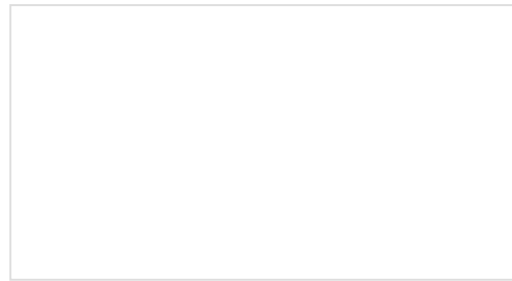
If any of these subjects sound unfamiliar, considering reading the following tutorials before continuing on.

- [What is an Arduino?](#) - New to Arduino? Start here.
- [Installing the Arduino IDE](#) - A refresher on using the Arduino environment.
- [Serial Peripheral Interface \(SPI\)](#) - This tutorial shows you how to communicate with an SD card over SPI.
- [SD Cards and Writing Images](#) - Learn the basics of SD and microSD cards.
- [Logic Levels](#) - Logic level basics for 3.3V and 5V systems.
- [How to Solder: Through Hole Soldering](#) - Basic information on how to solder to PTH pins.



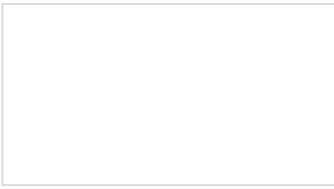
How to Solder: Through-Hole Soldering

This tutorial covers everything you need to know about through-hole soldering.



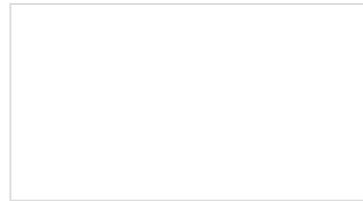
Serial Peripheral Interface (SPI)

SPI is commonly used to connect microcontrollers to peripherals such as sensors, shift registers, and SD cards.



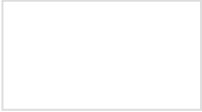
What is an Arduino?

What is this 'Arduino' thing anyway? This tutorial dives into what an Arduino is and along with Arduino projects and widgets.



Installing Arduino IDE

A step-by-step guide to installing and testing the Arduino software on Windows, Mac, and Linux.



Logic Levels

Learn the difference between 3.3V and 5V devices and logic levels.

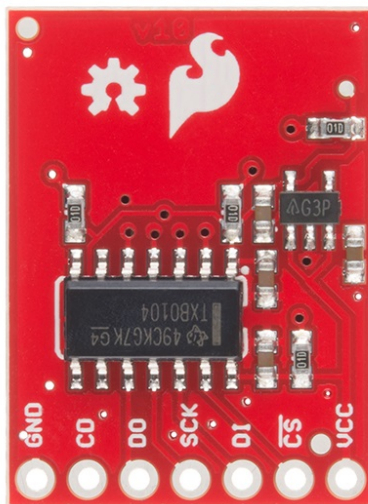


SD Cards and Writing Images

How to upload images to an SD card for Raspberry Pi, PCDuino, or your favorite SBC.

Hardware Overview

The [SparkFun Shifting microSD Breakout](#) is quite similar to the [SparkFun microSD Transflash Breakout](#), but with the additional feature of being 5.0V tolerant for ease of use. No more discrete level shifting is required! If you are using a 3.3V device, we recommend using the microSD Transflash Breakout without the level translation.



Back of the Level Shifting microSD Breakout

One thing that makes this product stand out is that it is `SPI_FULL_SPEED` stable. Some errors were seen in our testing with other products, but none have been caught using this board. We didn't have NIST do our testing. We can't rule out the influence of environmental factors or the host processor used, but if we were looking for stability and reliability at high speed, we'd use this board.

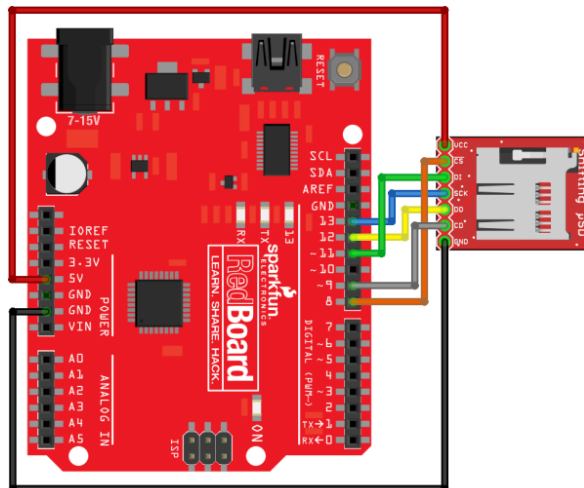
If your processor is capable of it, this board supports the use of even the fastest UHS μ SD cards. We only tested to 25MHz, but it should be good to two to four times that. Today most Arduino type μ Controllers are only capable of `SPI_HALF_SPEED` (6Mbps). Consider this board if you want a little future proofing or have a faster setup. The Arduino SD library is capable of `SPI_FULL_SPEED` (25Mbps).

The SparkFun Shifting μ SD is also a bit unique from its competitors in that it is bi-directional - it level translates all of its outputs back to the level of the hardware it's connected to.

Hardware Hookup

There are numerous ways to wire up an equivalent circuit to the one used in this guide. That will all depend on the type of Arduino you are using, the availability of a breadboard, or the types of wires you have laying around. The required materials list above assumes you have access to one of the most popular 5V Arduino form factors and you will wire it directly to the Shifting μ SD board with jumper wires. No breadboard required, and only minimal soldering to get a connection to the μ SD board.

Here is how you would wire up the Shifting μ SD to a 5V RedBoard or Arduino Uno ^[1].



Shifting μ SD board wired to 5V RedBoard

Warning! From the Arduino SPI library: If the SS pin ever becomes a LOW INPUT then SPI automatically switches to Slave, so the data direction of the SS pin MUST be kept as OUTPUT. This is pin 10, so be careful.

[1] **Note:** If you are using this with an Arduino Mega 2560, the SPI pins are broken out on a different location. You will need to wire the SPI pins differently on the Arduino Mega 2560.

Code Example

Note: If this is your first time using Arduino, please review our tutorial on [installing the Arduino IDE](#).

Copy and paste the code in the Arduino IDE. Selected the correct board (in this case, the **Arduino Uno**) and serial port that the Arduino enumerated to. Finally, hit the upload button.

```

/*
 * A simple file logger that allows the user to write to a file on the
 * µSD card
 * using the Arduino IDE Serial Monitor (57600 baud). Entered text is
 * written
 * to the µSD card every 20 characters, but to make sure everything is
 * written
 * append 'EOF' to your writing. Doing so writes everything remaining
 * in the
 * buffer to the file and reads back the contents of the file.
 */

#include <SPI.h>
#include <SD.h>

File fd;
const uint8_t BUFFER_SIZE = 20;
char fileName[] = "demoFile.txt"; // SD library only supports up to
8.3 names
char buff[BUFFER_SIZE+2] = ""; // Added two to allow a 2 char peek for
EOF state
uint8_t index = 0;

const uint8_t chipSelect = 8;
const uint8_t cardDetect = 9;

enum states: uint8_t { NORMAL, E, EOF };
uint8_t state = NORMAL;

bool alreadyBegan = false; // SD.begin() misbehaves if not first call

////////////////////////////////////
////////////////////////////////////
// Standard Arduino setup function
////////////////////////////////////
////////////////////////////////////
void setup()

```

```

{
  Serial.begin(57600);
  while (!Serial); // Wait for serial port to connect (ATmega32U4 type PCBAs)

  // Note: To satisfy the AVR SPI gods the SD library takes care of setting
  // SS_PIN as an output. We don't need to.
  pinMode(cardDetect, INPUT);

  initializeCard();
}

////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////
// Arduino calls this function over and over again when running
////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////
void loop()
{
  // Make sure the card is still present
  if (!digitalRead(cardDetect))
  {
    initializeCard();
  }

  if (Serial.available() > 0)
  {
    readByte();

    if (index == BUFFER_SIZE)
    {
      flushBuffer(); // Write full buffer to µSD card
    }
  }
}

```

```

////////////////////////////////////
////////////////////////////////////
// Do everything from detecting card through opening the demo file
////////////////////////////////////
////////////////////////////////////
void initializeCard(void)
{
    Serial.print(F("Initializing SD card..."));

    // Is there even a card?
    if (!digitalRead(cardDetect))
    {
        Serial.println(F("No card detected. Waiting for card.));
        while (!digitalRead(cardDetect));
        delay(250); // 'Debounce insertion'
    }

    // Card seems to exist. begin() returns failure
    // even if it worked if it's not the first call.
    if (!SD.begin(chipSelect) && !alreadyBegan) // begin uses half-spe
ed...
    {
        Serial.println(F("Initialization failed!"));
        initializeCard(); // Possible infinite retry loop is as valid as
anything
    }
    else
    {
        alreadyBegan = true;
    }
    Serial.println(F("Initialization done.));

    Serial.print(fileName);
    if (SD.exists(fileName))
    {
        Serial.println(F(" exists.));
    }
    else
    {
        Serial.println(F(" doesn't exist. Creating.));
    }
}

```

```

Serial.print("Opening file: ");
Serial.println(fileName);

Serial.println(F("Enter text to be written to file. 'EOF' will term
inate writing."));
}

////////////////////////////////////
////////////////////////////////////
// This function is called after the EOF command is received. It writ
es the
// remaining unwritten data to the µSD card, and prints out the full
contents
// of the log file.
////////////////////////////////////
////////////////////////////////////
void eof(void)
{
    index -= 3; // Remove EOF from the end
    flushBuffer();

    // Re-open the file for reading:
    fd = SD.open(fileName);
    if (fd)
    {
        Serial.println("");
        Serial.print(fileName);
        Serial.println(":");

        while (fd.available())
        {
            Serial.write(fd.read());
        }
    }
    else
    {
        Serial.print("Error opening ");
        Serial.println(fileName);
    }
}

```

```

    fd.close();
}

////////////////////////////////////
////////////////////////////////////
// Write the buffer to the log file. If we are possibly in the EOF st
ate, verify
// that to make sure the command isn't written to the file.
////////////////////////////////////
////////////////////////////////////
void flushBuffer(void)
{
    fd = SD.open(fileName, FILE_WRITE);
    if (fd) {
        switch (state) // If a flush occurs in the 'E' or the 'EO' stat
e, read more to detect EOF
        {
            case NORMAL:
                break;
            case E:
                readByte();
                readByte();
                break;
            case EO:
                readByte();
                break;
        }
        fd.write(buff, index);
        fd.flush();
        index = 0;
        fd.close();
    }
}

////////////////////////////////////
////////////////////////////////////
// Reads a byte from the serial connection. This also maintains the s
tate to

```

```
// capture the EOF command.
////////////////////////////////////
////////////////////////////////////
void readByte(void)
{
    byte byteRead = Serial.read();
    Serial.write(byteRead); // Echo
    buff[index++] = byteRead;

    // Must be 'EOF' to not get confused with words such as 'takeoff' or 'writeoff'
    if (byteRead == 'E' && state == NORMAL)
    {
        state = E;
    }
    else if (byteRead == 'O' && state == E)
    {
        state = EO;
    }
    else if (byteRead == 'F' && state == EO)
    {
        eof();
        state = NORMAL;
    }
}
}
```

The example code for this product is a simple file logger that allows the user to write to a file on the μ SD card using the [Arduino IDE Serial Monitor \(57600 baud\)](#). When the board boots you should see the following in the Serial Monitor:

```
Initializing SD card...Initialization done.
demoFile.txt doesn't exist. Creating.
Opening file: demoFile.txt
Enter text to be written to file. 'EOF' will terminate writing.
```

The last line of that block of text is important to note. Your work is written to the μ SD card every 20 characters, but to make sure everything is written append EOF to your writing. Doing so writes everything remaining in the buffer to the file and reads back the contents of the file.

```
EOF
```

```
demoFile.txt:
```

```
Test line of text.
```

```
Bacon ipsum dolor amet beef picanha drumstick alcatra brisket, short  
ribs sirloinBacon ipsum dolor amet beef picanha drumstick alcatra bris  
ket, short ribs sirloin.
```

One thing to note is that the UART buffer on the Arduino might limit the number of characters entered on a single line. In one test I noticed that my Arduino only accepted 164 bytes before losing data, but I've seen that vary a bit.

Troubleshooting

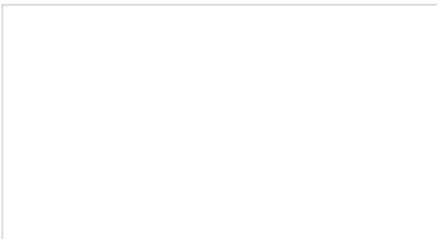
Arduino has troubleshooting tips on their [Notes on the Arduino SD Card Library](#) page, including instructions for formatting new cards (if needed) and using the library with other Arduino boards.

Resources and Going Further

Now that you know how to add more storage to your Arduino projects, it's time to get out there and create something amazing. Need some inspiration? Check out these other SparkFun tutorials.

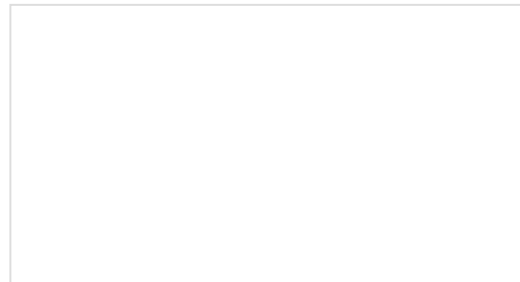
- SD cards are a great way to add more storage to Internet of Things projects. You can add a microSD Shield to a [WiFly Shield](#) to serve up larger web pages or hold more data. You can also use the [CC3000 Shield](#), which has a microSD card slot built in.
- Need more power than your Arduino can provide? Check out the [Edison](#), which also has a [microSD card Block](#) to add more storage to larger projects.
- Logging data is a common use for SD cards. Take your logging project to the next level with the [Logomatic](#).

Or check out any tutorial tagged with [logging](#) for inspiration:



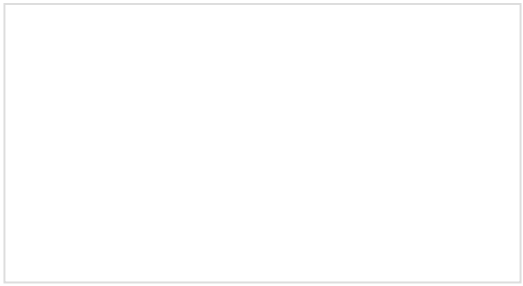
Blynk Board Project Guide

A series of Blynk projects you can set up on the Blynk Board without ever re-programming it.



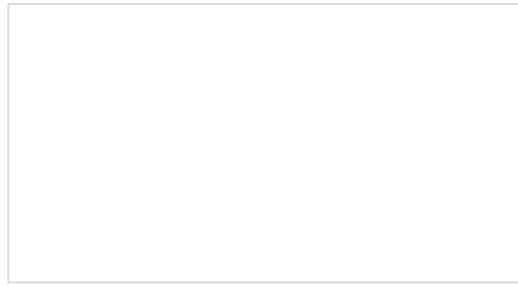
Logging Data to Google Sheets with the Tessel 2

This project covers how to log data to Google Sheets two ways: using IFTTT with a web connection or a USB pen drive and "sneakernet" without.



Wireless Remote Weather Station with micro:bit

Monitor the weather without being exposed to it through wireless communication between two micro:bits using the radio blocks! This is useful if your weather station is installed in a location that is difficult to retrieve data from the OpenLog. We will also explore a few different ways to send and receive data.



Monitor Sensor Data from Anywhere

Using WiFi to send sensor data from an ESP32 to a WiFi network and be able to read it from an IoT Dashboard from anywhere in the world.